# Campus Carpool Matching Platform

Graphic Era Hill University, Dehradun — Phase-1 (MVP) Engineering Playbook

Architecture: Polyglot Monorepo | Model: 4 Persistent Git Branches | Target: 10-Week Sprint | Status: Milestone 1 Kickoff

## 1. The 4-Branch Persistent Team Workflow

To eliminate branch clutter and avoid creating throwaway branches for every mini-task, our team works with exactly **4 persistent developer branches** alongside `main`.

| `main` | `akshit` | `teammate-2` | `teammate-3` |
|---|---|---|---|
| **Protected**: Production-ready code only. Never commit directly. | Team Lead / Fullstack Integration & Architecture. | Persistent branch for Teammate 2. | Persistent branch for Teammate 3. |

> **The Ironclad Rule: Never Break "main"**
>
> `main` must be stable and demo-ready 24/7. When your teammate pulls from `main`, they should get code that starts and runs without runtime crashes. No incomplete or untested code gets merged into `main`.

### The 5-Step Daily Engineering Cycle

#### Step 1: Start your day by syncing the latest main into your personal branch

```
# Switch to your persistent branch
git checkout akshit
# Pull the latest tested work from main
git pull origin main
```

#### Step 2: Code your assigned task locally

Work inside your designated folder (`backend/`, `client/`, or `ml-service/`). Verify the feature works on your machine.

#### Step 3: Execute the Mandatory Pre-PR Verification Gate

Run local builds and verify zero runtime errors before pushing. See verification checklist below.

#### Step 4: Push to your personal branch & open a Pull Request (PR)

```
git status
git add .
git commit -m "feat(auth): add JWT verification middleware and test endpoint"
git push origin akshit
```

#### Step 5: Code Review & Merge into main

Open a PR on GitHub (`akshit → main`). Another teammate verifies the change. Once merged, notify the team so they can run `git pull origin main` into their respective branches.

## 2. Pre-PR Quality & Testing Checklist

| Area | Verification Requirement | How to Verify |
| --- | --- | --- |
| **Build & Syntax** | Zero syntax errors, unhandled exceptions, or missing dependencies. | `npm run dev` starts clean without errors. |
| **Database Integrity** | All queries respect soft-deletes and foreign keys. | Check Supabase SQL editor; test both valid & invalid inputs. |
| **Security & Secrets** | Zero hardcoded API keys, JWT secrets, or DB passwords. | Inspect `git diff`. Ensure `.env` is never staged. |
| **Edge Cases** | Fails gracefully on invalid input (returns 400, not 500 server crash). | Test with missing body parameters / malformed inputs. |

## 3. Phase-1 Granular Implementation Roadmap

Milestone 1: Authentication & Institutional Verification The Foundation

### Task 1.1: Backend Supabase Client & JWT Auth Middleware

BACKEND

Initializes Supabase Admin client in Express and implements `verifyAuth` middleware that validates incoming Bearer JWTs and attaches `req.user`.

**Definition of Done:**

- `backend/src/config/supabase.js` connects via Service Role Key.
- `verifyAuth` extracts token, calls Supabase Auth, fetches row from `users`.
- Unauthenticated requests receive clean `401 Unauthorized` response.

### Task 1.2: Client Auth Provider & Supabase Client Setup

FRONTEND

Sets up Supabase JS in React and builds a global `AuthContext` that persists session state and provides auth methods across the app.

**Definition of Done:**

- `client/src/services/supabase.js` configured with public anon key.
- `AuthContext.jsx` provides `user`, `session`, `login`, `signup`, `logout`.
- JWT automatically attached to backend API requests.

### Task 1.3: University Signup & Login UI (@gehu.ac.in restriction)

FULL-STACK

Login and signup screens with strict institutional email domain filtering against `universities.email_domain`.

**Definition of Done:**

- Rejects any signup not ending in `@gehu.ac.in`.
- Creates matching profile row in `users` table with `verification_status: 'pending'`.
- User routed to Pending Verification holding screen upon initial registration.

### Task 1.4: ID Card Upload to Private Supabase Storage

FULL-STACK

Direct-to-storage upload for student ID card images into the private `id-cards` bucket, bypassing backend file proxying.

**Definition of Done:**

- Max size 5MB, JPG/PNG only.
- Saves storage path in `users.id_card_image_url`.
- Private bucket ensures image cannot be accessed via direct public URL.

### Task 1.5: Admin Verification Queue & 5-Min Signed URLs FULL-STACK

Role-gated portal (`role === 'admin'`) displaying pending students with temporary signed URLs to inspect ID cards.

**Definition of Done:**

- `GET /api/v1/admin/verifications/pending` lists pending students.
- Generates signed URL with 5-minute validity on review modal open.
- `PATCH /api/v1/admin/verifications/:id/approve` updates status to `verified`.

## Milestone 2: Vehicle Management & Ride Posting Driver Flow

### Task 2.1: Vehicle Registration & Seat Capacity Derivation FULL-STACK

Register vehicles (bike, scooty, car). Seat capacity automatically derives from vehicle type (bike/scooty: 1, car: 4, editable).

**Definition of Done:**

- `POST /api/v1/vehicles` and `GET /api/v1/vehicles/mine`.
- Deletion is blocked if vehicle is linked to any active ride (posted or departed).

### Task 2.2: "Offer a Ride" Ad-Hoc Posting Engine FULL-STACK

Verified drivers post an ad-hoc ride with origin address, departure time, seats, optional price, and gender preference.

**Definition of Done:**

- Origin and fixed campus destination saved as PostGIS `GEOGRAPHY(Point, 4326)`.
- Requires verified DL number on profile before allowing post.

## Milestone 3: Search, Detour Filter & ML Matching Rider Flow

### Task 3.1: Python ML Microservice (FastAPI + Sentence-Transformers) ML MICROSERVICE

Microservice running `all-MiniLM-L6-v2` computing cosine similarity between typed pickup text and candidate origin texts.

**Definition of Done:**

- `POST /embed-similarity` accepts query & candidates, returns scores (0 to 1).
- Successfully scores "Subhash Nagar Rd" vs "Subhash Nagar Road" as strong match.

### Task 3.2: Spatial Detour Filter & Mutual Gender Checks BACKEND / DB

Express search constructs PostGIS spatial query checking `ST_DWithin`, departure time window, and mutual gender preferences.

**Definition of Done:**

- Spatial query utilizes GiST index for fast detour calculation.
- Mutual gender filter excludes incompatible pairs symmetrically.

### Task 3.3: 50/50 Weighted Fusion Ranking Engine & Fallback BACKEND

Ranks surviving candidates: `final_score = (0.5 × rating) + (0.5 × price)`. If ML microservice times out (>2s), falls back to Levenshtein string distance.

**Definition of Done:**

- Search never fails or hangs even if Python service is stopped.
- Returns ranked list of top candidate cards.

### Task 3.4: "Find a Ride" Search Screen & Ranked Cards UI FRONTEND

Search interface with pickup location, departure time, and ranked result cards showing rating, price, and detour proximity.

**Definition of Done:**

- Shows ranked cards (no map in Phase-1).
- Offers transparent toggle to view excluded rides if gender filter returns zero matches.

## Milestone 4: Ride Request State Machine & Atomic Concurrency Concurrency Engine

### Task 4.1: Parallel Request Dispatch & Dynamic Expiry FULL-STACK

Rider can select up to 5 drivers in parallel. Dynamic expiry calculated as `now() + fraction × time_to_departure`.

**Definition of Done:**

- App enforces maximum 5 parallel active requests per rider.
- Partial unique index prevents duplicate requests to the same ride.

### Task 4.2: Atomic First-Accept-Wins Transaction (Row-Level Lock)

BACKEND

When Driver A accepts, locks row via `SELECT ... FOR UPDATE`, sets request to accepted, and auto-cancels rider's other 4 requests.

**Definition of Done:**

- Guarantees two drivers cannot simultaneously accept the same rider.
- Auto-cancelled drivers receive instant Socket.IO cancellation notification.
- Explicit driver decline instantly frees rider's request slot.

## Milestone 5: Real-Time Active Ride (Chat & Live GPS) Socket.IO Layer

### Task 5.1: Socket.IO Server & Room Lifecycle

BACKEND

Room management for ride rooms (`ride_<id>`) and personal user notification rooms (`user_<id>`).

**Definition of Done:**

- Always-write (to notifications table) then always-emit pattern.
- `GET /api/v1/notifications?unread=true` covers offline catch-up.

### Task 5.2: In-App Coordination Chat (Pre-Departure)

FULL-STACK

Real-time messaging between driver and rider to verbally clarify landmarks before heading out.

**Definition of Done:**

- Persisted in `messages` table, broadcast live over Socket.IO room.
- Access strictly restricted to the two ride participants.

### Task 5.3: Ephemeral Live GPS Relay & Driver Safety UX

FULL-STACK

Driver marks "Departed", live GPS emits via `navigator.geolocation`. Relay only; not stored in database in Phase-1.

**Definition of Done:**

- Driver UI minimizes distraction (no forced full-screen map glancing while driving).
- Relay stops immediately when ride is completed.

### Task 5.4: Arrival & No-Show Incident Handling

FULL-STACK

Driver manual trigger if rider is missing at pickup point. Soft UI hint for wait time (no forced clock).

**Definition of Done:**

- Sets status to `no_show` and logs row in `incident_logs`.
- Penalizes rider's reliability score; driver score remains clean.

## Milestone 6: Completion, Ratings & Peer UPI Facilitation Closure & Trust

### Task 6.1: Driver-Only Completion & Peer-to-Peer UPI QR Code

FULL-STACK

Single-tap completion by driver triggers UPI payment QR code directly linking to driver's UPI ID. Zero gateway liability.

**Definition of Done:**

- Payment triggered ONLY after ride completion.
- App displays UPI QR and deep link directly to driver bank-to-bank.

### Task 6.2: Post-Ride Rating & Decimal Score Recalculation FULL-STACK

Post-ride 1-5 star review submitted. Application code recalculates exact decimal `composite_rating` on `users` table.

**Definition of Done:**

- Rating modal appears automatically on rider client upon completion.
- Composite score factors in explicit stars and incident history (no-shows, late cancellations).

## Milestone 7: Integration Testing & Production Deployment Live Pilot Launch

### Task 7.1: End-to-End Lifecycle Scenario Testing TESTING

Execute 12 verification test scenarios (Happy path, parallel request race conditions, ML microservice down fallback, vehicle deletion guard).

### Task 7.2: Free-Tier Production Deployment DEVOPS

Vercel (React PWA) + Render (Node.js & FastAPI ML Service) connected to live hosted Supabase.

**Immediate Assignment for Today (Team Lead: Akshit)**

Execute **Task 1.1: Backend Supabase Client & JWT Auth Middleware** on branch `akshit`. Once verified and merged into `main`, teammates pull `main` to build Task 1.2, 1.3, and 3.1 in parallel!